



나

A NEW PATTERN

Karpathy의 LLM Wiki

지식을 컴파일하는 새 패턴

RAG를 넘어서, 지루한 일은 LLM에게



지식 베이스

매번 다시 읽지 않는다
한 번 컴파일하고 누적한다

오늘 한 문장만 가져간다면

강의 60분이 끝나도 손에 남는 한 줄



LLM Wiki는 검증 대역폭을 **재배치** 한다 — 어디로 재배치할지 당신이 결정한다.

Q1

왜 RAG로는 부족한가?

매번 처음부터 읽는 비용

Q2

LLM Wiki는 정확히 무엇인가?

컴파일러로 사고하기

Q3

어디에 의심을 던질 것인가?

패턴의 빈 곳

RAG는 매번 처음부터 읽는다

누적되지 않는 비용 · 인간 위키가 다 죽은 이유



DAC

학생이 이미 해본 그것

PDF 업로드

→

질문

→

답변

같은 PDF를 100번 물어보면 100번 다시 읽는다.

비용 구조	$O(\text{쿼리 수} \times \text{문서 크기})$
누적 산출물	없음
5개 문서 합성	매번 5조각 재조립

"Nothing is built up."

— Andrej Karpathy



시

그런데 — 인간 위키는 왜 다 죽었나?

Wikipedia 빼고 회사 위키 · 개인 노트 대부분은 죽는다. 죽는 이유는 읽거나 생각이 아니라 — 유지보수의 지루함.

cross-reference 갱신, 요약 최신화, 모순 표기, 일관성 유지. 이 모든 복키핑 비용이 위키가 주는 가치보다 빨리 커진다.

MEMEX · 1945

"Bush가 못 푼 문제는 누가 유지보수를 하느냐였다."

컴파일러로 사고하기

RAG는 인터프리터, LLM Wiki는 컴파일러



학생이 이미 아는 답

왜 우리는 코드를 매번 인터프리트하지 않고
컴파일하는가?

→ 같은 일을 반복하지 않으려고.

	RAG	LLM Wiki
비유	인터프리터	컴파일러
출력	매번 새 답변	영속 markdown 위키
비용 위치	쿼리 시점	Ingest 시점

"compiled once and kept current."

— Andrej Karpathy



컴파일러 → LLM Wiki 매핑

Compiler	LLM Wiki
Source code (.c, .py)	Raw sources (PDF, 메모)
Frontend (parse → AST)	LLM이 읽고 entity 추출
IR / AST	Wiki markdown
Symbol table	index.md
Build log	log.md
Optimizer (DCE, CSE)	Lint pass
Codegen (per query)	Answer synthesis

정직 ✕

컴파일러는 deterministic, LLM은 아니다. 이 깨짐이 강의 후반 비판층의 씨앗.

3-Layer Separation

Raw / Wiki / Schema — 누가 무엇을 쓰는가



Schema

LAYER 3

Schema (AGENTS.md)

위키를 어떻게 관리할지의 컨벤션 — 자연어로 쓴 코드



Wiki

LAYER 2

Wiki (markdown)

entity · concept · 요약 페이지 — 한 ingest에 10~15장 갱신



Raw

LAYER 1

Raw sources

PDF · 웹클리핑 · 트윗 · 미팅 메모 — 불변 source of truth

Layer

누가 쓴다

규칙

Raw

인간만

불변

Wiki

LLM만

한 ingest 10~15 페이지



그

왜 Schema가 진짜 핵심인가

**AGENTS.md = LLM에게 주는
style guide + CONTRIBUTING.md**

페이지 형식 컨벤션, ingest 워크플로우, 모순 처리 규칙을 **자연어로** 적은 문서. LLM이 이걸 **deterministic하게** 따른다 — 그러므로 이건 **프로그램**이다.



Karpathy의 **Software 3.0** 이 가장 구체적으로 실현된 사례.

CS 비유 1

OS user/kernel mode

CS 비유 2

git working tree / index /
commit

시스템처럼 운영된다

Ingest / Query / Lint + index.md · log.md = 부트로더



Ingest

Ingest

자료 drop → LLM 읽기 → 요약 → **index 갱신** →
entity 페이지 갱신 → log 추가

한 번에 10~15 페이지를 touch한다



Query

Query

index.md → 관련 페이지 read → **합성** → 인용 →
markdown · 표 · 차트 등 다양한 형식

★ 좋은 답변은 다시 위키 페이지가 된다 — 탐색
이 누적된다



Lint

Lint

주기적: **모순** · stale claim · orphan 페이지 · 누락
cross-reference 청소

DB의 VACUUM, 메모리의 GC와 같은 자리



메타 파일 두 개가 시스템의 부트로더다 — 없으면 그냥 markdown 폴더

index.md

컨텐츠 카탈로그

≈ DB catalog table

log.md

append-only 이벤트 스트림

≈ WAL (Write-Ahead Log)

```
## [2026-05-16] ingest | Karpathy LLM Wiki gist
## [2026-05-16] query | "RAG와 차이가 뭐냐"
```

```
## [2026-05-16] lint | 3 orphan pages cleaned
```

이 패턴은 어디까지 가는가

"Embedding 필요 없다"의 진짜 의미 · 생태계 안에서의 위치



디자인 선택으로 인식하라

규모	적정 도구
~100 sources	LLM Wiki (vector DB 불필요)
1,000+ sources	Vector DB / Graph DB → 다른 아키텍처
Enterprise (10,000+)	RAG + 거버넌스 + 정밀 검색

이건 **원칙** 이 아니라 **디자인 선택** 이다.

LLM Wiki는 **personal-scale KM**의 답이지 enterprise KM의 답이 아니다.



단독 발명이 아니다 — 더 큰 흐름의 인스턴스

2024 ~ 2026, LLM-friendly documentation 표준화의 흐름

표준	날짜	역할
llms.txt	2024-09	웹사이트 → LLM에게 주는 markdown (Jeremy Howard)
MCP	2024-11	런타임 컨텍스트 전달 프로토콜 (Anthropic)
AGENTS.md	2025-08	크로스 톨 표준 (LF 기준 2025-12)
Context Eng.	2025-09	compaction · structured note-taking (Anthropic · LangChain)
LLM Wiki	2026-04	위 흐름을 워크플로우로 묶어낸 패턴 (Karpathy)

"한 명의 천재적 발견"이 아니라
집단적 흐름의 한 시점 정리다.

어디에 의심을 던질 것인가



성숙한 엔지니어는 패턴을 의심한다 — 4가지 미해결 가정

#	Karpathy의 주장	의심할 지점	실전 시사점
1	"LLMs don't forget cross-references"	컨텍스트 윈도우 밖 페이지 reference는 silent failure . Make처럼 의존성 그래프가 명시적이지 않다.	Lint pass는 옵션이 아니라 필수
2	"Markdown is easy for LLMs"	형식과 의미는 다르다. AGENTS.md는 형식만 강제. 같은 entity를 다른 페이지에서 다르게 합성 가능.	Schema는 형식만 보장. 의미는 lint + human audit
3	Raw가 외부에서 갱신되면?	Supersession 메커니즘이 비어있다 . v2 fork의 시간 기반 decay는 같은 실수를 반복하는 길 .	명시적 supersession이 정답. Schema에 raw 변경 이벤트 규칙을 써줘야 한다
4	패턴 자체의 거버넌스	엑세스 컨트롤·버전 관리·품질 지표·audit log 모두 없다 . 개인은 OK, 팀·민감 정보는 치명적.	LLM Wiki는 대체가 아니라 부트스트랩. 위에 거버넌스 레이어를 얹어라



v2 비판자들의 공통 지적: "LLMs treated as reliable" — 이 가정이 가장 큰 미해결 문제다.

검증 대역폭을 재배치한다

패턴은 검증을 자동화하지 않는다 — 검증할 대상의 형태를 바꿀 뿐이다



AI 속도가 아니라 **검증 대역폭** 이 품질의 상한이다.



하

확장 방향 ↑

검증 표면이 응축된다

검증 대상이 영속·diff-able한 markdown으로 **압축**된다.

- ① Schema (소수, 안정)
- ② Wiki 페이지 (영속, diff)
- ③ log.md (시간순, 감사 가능)

→ 인간 1명이 더 많은 정보를 검증 가능



대

몰래 축소 ↓

위키 자체는 누가 검증?

RAG는 매 응답마다 검증 기회가 있었지만, Wiki는 **ingest 시점**에만 있다.

- 틀린 합성이 영속화
- 잘못된 합의가 정설처럼 굳음
- 검증 깊이 요구는 ↑

→ 검증 게을리하면 RAG보다 빨리 오염



결 LLM Wiki는 검증 대역폭을 재배포한다 —
어디로 재배포할지 당신이 결정한다.

5개 기둥 회수 · 80년 만의 실현

강의가 끝나도 남는 한 문장



5개 개념 기둥

#	기둥	CS 비유
1	Persistence vs Recomputation	DP / Memoization
2	3-Layer Separation	user/kernel 권한 분리
3	Incremental Maintenance is Killer	Make / Bazel
4	Schema as Code (Software 3.0 DSL)	CONTRIBUTING.md
5	Index.md + Log.md = Bootloader	DB catalog + WAL



Memex — 80년 만의 실현

1945

Vannevar Bush, "As We May Think": 개인이 모든 자료를 연결해서 보관하는 **Memex**. 그가 못 푼 문제는 누가 **cross-reference**를 유지하는가.

"The part he couldn't solve was who does the maintenance. The LLM handles that."

— Andrej Karpathy, 2026



강의가 끝나도 남는 한 문장

LLM Wiki는 검증 대역폭을 재배포한다 — 어디로 재배포할지 당신이 결정한다.